

ThreadLocal之一



引入ThreadLocal的必要性

北京理工大学计算机学院
金旭亮

背景



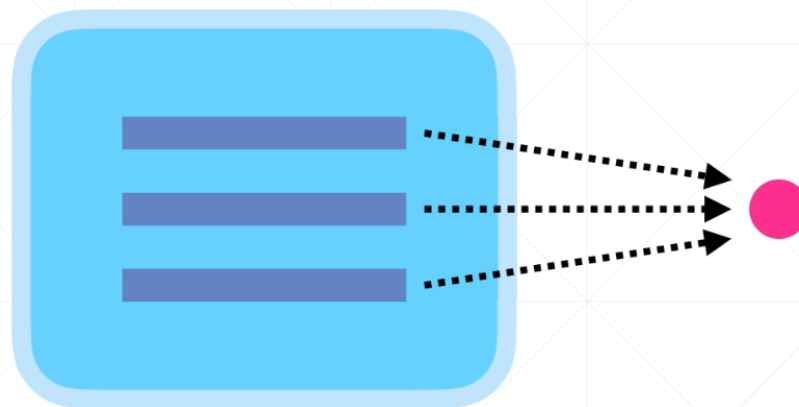
多线程访问同一个数据对象，是引发错误的原因之一。

```
public class MyData {  
    private int value;  
    public int getValue() {  
        return value;  
    }  
    public void setValue(int value) {  
        this.value = value;  
    }  
}
```

MyThread线程在运行时，会将MyData对象的value字段值加一。

```
public class MyThread extends Thread{  
    private final MyData myData;  
    public MyThread(MyData myData) {  
        this.myData = myData;  
    }  
    @Override  
    public void run() {  
        try {  
            Thread.sleep( millis: 500);  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
        myData.setValue(myData.getValue()+1);  
    }  
}
```

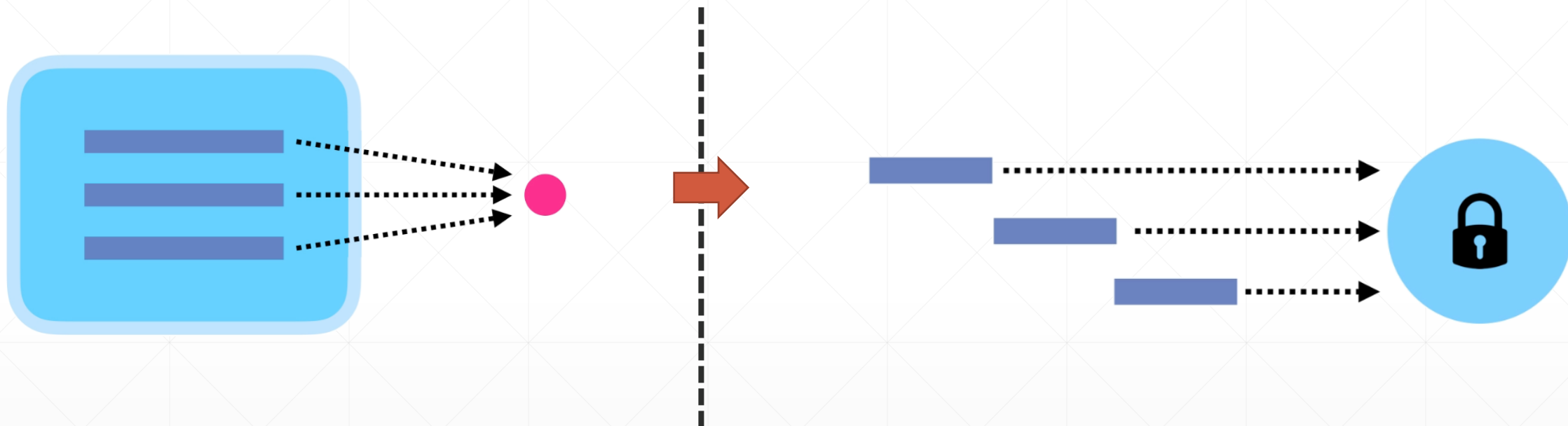
```
private static void wrongCode() {  
    List<Thread> threads = new ArrayList<>();  
    var myData = new MyData();  
    int threadCount = 5;  
    for (int i = 0; i < threadCount; i++) {  
        var thread = new MyThread(myData);  
        threads.add(thread);  
        thread.start();  
    }  
    threads.forEach(thread -> {  
        try {  
            thread.join();  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    });  
    System.out.println(myData.getValue());  
}
```



示例程序创建了5个线程访问同一个MyData对象，每个线程都将其value字段值加一。

正确的结果应该是5，但实际上，每次运行时结果可能都不一样。

修正错误



多个线程存取同一个共享数据对象，是导致数据存取错误的关键之一，可以通过让多个线程顺序访问的方式解决这个问题，编写“同步方法”或“同步代码段”，是一种常见的技术解决方案。

清除Bug



使用右图所示的类，取代原先的MyThread，可以解决数据出错的问题。

```
1 usage
public class MyThreadUseSynchronize extends Thread {
    4 usages
    private final MyData myData;
    1 usage
    public MyThreadUseSynchronize(MyData myData) { this.myData = myData; }
    @Override
    public void run() {
        try {
            Thread.sleep( millis: 500 );
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        //使用同步代码块，独占访问MyData
        synchronized (myData){
            myData.setValue(myData.getValue()+1);
        }
    }
}
```



前面的示例中多个线程访问同一数据对象导致数据存取错误，我们说可以使用同步方法或同步代码块解决问题，但这个方案的成本比较高，那除此之外，还有没有一种“轻量级”的方法，不使用`synchronized`也能解决问题？



解答：完全可以，只要让每个线程都访问独立的对象实例就行了。

修改示例

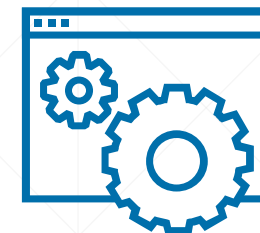
让每个线程都独占访问单独的对象，就不可能会引发数据存取错误。



等待所有线程运行结束，再访问所有数据对象，统计结果，就不会出错了，肯定是5，因为五个线程“井水不犯河水”，彼此隔离。

```
private static void correctCode() {
    List<Thread> threads = new ArrayList<>();
    List<MyData> myDataList = new ArrayList<>();
    int threadCount = 5;
    for (int i = 0; i < threadCount; i++) {
        var myData = new MyData();
        myDataList.add(myData);
        var thread = new MyThread(myData);
        threads.add(thread);
        thread.start();
    }
    //等待5个线程运行结束
    threads.forEach(thread -> {
        try {
            thread.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    });
    //访问所有数据对象，进行统计，获取最终结果
    var result :int = myDataList.stream().mapToInt(MyData::getValue).sum();
    System.out.println(result); //结果一定是 5
}
```

创建五个线程，
每个线程分配五个“独享”对象



由于“让每个线程独占访问专有对象”是一个比较普遍的作法，为此，JDK提供了一个`ThreadLocal<T>`类型，可以方便地帮助我们做到这一点.....

ThreadLocal示例



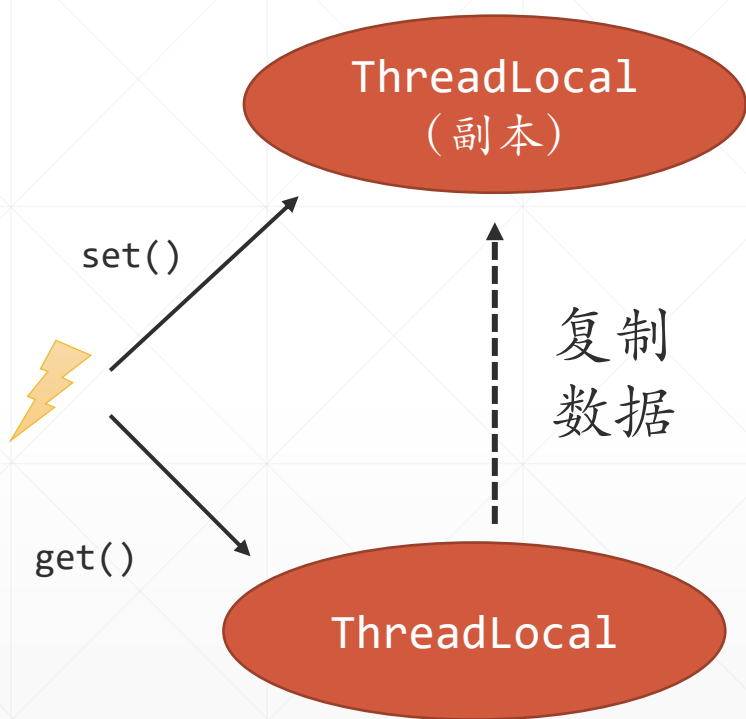
使用ThreadLocal定义变量，每个线程都可以调用ThreadLocal.set()方法向此变量中保存数据，而同一个线程再调用get()方法时，就会取出来原先放入的数据，各个线程互不影响。

```
Run: ThreadLocalTest x
"C:\Program Files\Java\jdk-15\bin\java.exe"
Thread-1产生了数据: 231355449
Thread-0产生了数据: 1412511705
Thread-0:1412511705
Thread-1:231355449
Process finished with exit code 0
```

从左图可以看到，每个线程都访问自己独享的数据，别的线程访问不到，因此可以不必加锁，从而避免了加锁带来的性能损失。

示例：ThreadLocalTest.java

ThreadLocal示例



```
public class ThreadLocalTest {  
    //放在外部的ThreadLocal  
    static ThreadLocal<Integer> tl = new ThreadLocal<>();  
    public static void main(String[] args) {  
        //创建两个线程  
        for (int i = 0; i < 2; i++) {  
            new Thread(() -> {  
                String threadName = Thread.currentThread().getName();  
                int dataValue = new Random().nextInt();  
                System.out.println(threadName + "产生了数据: " + dataValue);  
                tl.set(dataValue);  
                //线程内部new一个对象, 通过它来访问数据  
                new MyClass().visitData();  
            }).start();  
        }  
        private static class MyClass {  
            public void visitData() {  
                String threadName = Thread.currentThread().getName();  
                Integer data = tl.get();  
                System.out.println(threadName + ":" + data);  
            }  
        }  
    }  
}
```